

EduStack Masterclass

Your Ultimate Computer Science & Engineering Hub

VOLUME 2 — Authentication, Security & Payment Cryptography

Tier-1 Interview Reference Guide (Visa, Amazon, Oracle, JPMC, Microsoft)

JWT Cookies | Google OAuth 2.0 | bcrypt Salt | HMAC Verification | 25 Tier-1 Q&As

Developer:	Shubham Kumar CSE Student NIT Patna
Target Roles:	Security Engineer, SDE II Backend, Payment Systems Architect
Security:	13 Security Layers: Helmet, NoSQL Sanitization, Rate Throttling, HMAC-SHA256
Authentication:	JWT stored in httpOnly, Secure, SameSite cookies + OTP Email Verification
Third-Party:	Google OAuth 2.0 via Passport.js + Dynamic Account Linking Strategy
Payment:	Razorpay Gateway with Server-side Order Initialization & timingSafeEqual
Volume 2:	Auth Engineering, Security Mitigation, Payment Cryptography & 25 Q&As

Table of Contents — Volume 2

- 7. Authentication Architecture - Local Auth & JWT**
Stateless JWT in OWASP httpOnly cookies vs localStorage
- 8. Cryptographic Foundations & Attack Mitigations**
bcrypt 12 rounds, HMAC-SHA256, timing side-channels
- 9. Google OAuth 2.0 Integration & Account Linking**
OAuth 2.0 code grant flow, Passport.js internals, account linking
- 10. 13-Layer Defense-in-Depth Security Framework**
Helmet security headers, NoSQL injection prevention, anti-enumeration
- 11. Razorpay Payment Gateway & Cryptographic Verification**
3-step payment flow, HMAC signature verification, timingSafeEqual
- 12. Tier-1 Interview Q&A - Auth & Security (25 Q&As)**
Deep questions asked by Visa, Amazon, Oracle, JPMC, Microsoft

Authentication Architecture - Local Auth & JWT

OWASP httpOnly cookie security, stateless JWT verification, and payload scoping

7.1 Stateless JWT vs Stateful Sessions

Authentication in web architectures is fundamentally a choice between stateful server-side sessions and stateless cryptographic tokens. Stateful session architectures require querying a centralized session store (Redis or DB) on every request, creating a single point of failure and bottleneck for horizontal scaling.

EduStack implements stateless JSON Web Tokens (JWT) stored exclusively in httpOnly, Secure, SameSite cookies. The server validates the token mathematically using HMAC-SHA256 signature verification, eliminating database reads for session validation while protecting tokens against Cross-Site Scripting (XSS) theft.

Property	localStorage JWT	httpOnly Cookie JWT (EduStack)
JavaScript Accessible	Yes (localStorage.getItem()) - Vulnerable to XSS	No - Completely invisible to browser JavaScript engine
XSS Exposure	CRITICAL - Script injection steals user tokens	SECURE - XSS scripts cannot read httpOnly cookie
CSRF Vulnerability	Low (header required)	Mitigated by SameSite=Lax/None + CORS whitelist
Auto-Transmission	Manual (JS attaches Authorization header)	Automatic by browser engine on matching origin
Expiry Control	Client JS managed	Server enforced via Max-Age and Set-Cookie flags

Security & Cryptography Code Implementation

```
// Generating and Attaching OWASP-Compliant httpOnly JWT Cookie
const jwt = require('jsonwebtoken');

const attachCookieToken = (res, userId) => {
  const token = jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.cookie('edustack_token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'lax',
    maxAge: 7 * 24 * 60 * 60 * 1000
  });
  return token;
};
```

Cryptographic Foundations & Attack Mitigations

bcrypt 12 salt rounds, timing side-channel attacks, and timingSafeEqual

Security & Cryptography Code Implementation

```
// Cryptographic Password Hashing in Mongoose Pre-Save Hook
userSchema.pre('save', async function(next) {
  if (!this.isModified('password') || !this.password) return next();
  if (this.password.startsWith('$2a$') || this.password.startsWith('$2b$')) return next();
  const salt = await bcrypt.genSalt(12);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

Razorpay Payment Gateway & Cryptography

Server-side order creation, HMAC-SHA256 verification, and timingSafeEqual

PAYMENT & AUTH FLOW: Razorpay Cryptographic Verification Lifecycle

Stage 1: Step 1: POST /api/payments/create-order -> Server calls Razorpay API -> Saves Payment doc (status: created)



Stage 2: Step 2: Client renders Razorpay Checkout SDK -> User completes payment -> SDK receives orderId, paymentId, signature



Stage 3: Step 3: POST /api/payments/verify -> Server recomputes HMAC-SHA256(orderId + "|" + paymentId, KEY_SECRET)



Stage 4: Step 4: Server executes crypto.timingSafeEqual(expected, received) -> Sets User.isPremium = true

Security & Cryptography Code Implementation

```
// Cryptographic Payment Verification Service
const crypto = require('crypto');

const verifyPaymentSignature = (orderId, paymentId, signature) => {
  const message = `${orderId}|${paymentId}`;
  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(message)
    .digest('hex');

  try {
    return crypto.timingSafeEqual(
      Buffer.from(expectedSignature, 'hex'),
      Buffer.from(signature, 'hex')
    );
  } catch {
    return false;
  }
};
```

Tier-1 Interview Q&A - Auth & Security

25 Deep Technical Questions asked by Visa, Amazon, Oracle, JPMC, Microsoft

Q: What is a timing attack and how does `crypto.timingSafeEqual` prevent it during payment verification?

Comprehensive Security Answer:

A timing attack is a side-channel attack where an attacker measures microsecond variations in server response time to guess secret values. JavaScript's default string equality operator (`===`) short-circuits - it returns false as soon as it encounters the first non-matching byte. `crypto.timingSafeEqual` executes in constant time regardless of where byte mismatches occur.

- > Standard string comparison (`===`) reveals prefix match length through CPU execution time.
- > `crypto.timingSafeEqual` compares all buffer bytes unconditionally in constant execution time.
- > Essential for HMAC-SHA256 signature verification in payment gateways like Visa, PayPal, and Razorpay.

Q: How do you prevent NoSQL injection in MongoDB when handling JSON input?

Comprehensive Security Answer:

NoSQL injection occurs when attackers send MongoDB query operators (e.g., `{"$gt": ""}`) in JSON request bodies. `express-mongo-sanitize` strips all `$` and `.` prefix characters from `req.body`, `req.query`, and `req.params` before route execution.

- > Attack payload: `{"email": {"$gt": ""}, "password": {"$gt": ""}}` bypassing login.
- > `express-mongo-sanitize` strips `$` and `.` characters across all inputs.